conjuntos de tarefas aceitos nesses testes certamente são escalonáveis; porém entre os descartados podem existir conjuntos escalonáveis.

- *Testes necessários* correspondem também a análises simples porém não tão restritivas. O fato de um conjunto ter passado por um teste necessário não implica que o mesmo seja escalonável. A única garantia que esse tipo de teste pode fornecer é que os conjuntos descartados de tarefas certamente são não escalonáveis.

A figura 2.4 ilustra os três tipos de testes descritos acima.

A *utilização de uma tarefa* T_i que serve como uma medida da ocupação do processador pela mesma, é dado por:

$U_i = C_i / P_i$ se a tarefa T_i é periódica, ou
$U_i = C_i / Min_i$ se a tarefa T_i é esporádica,

onde C_i, P_i e Min_i são respectivamente o tempo máximo de computação, o período e o intervalo mínimo entre requisições da tarefa T_i. A utilização de um processador (U) dá a medida da ocupação do mesmo por um conjunto de tarefas $\{ T_1, T_2, T_3, ..., T_n\}$:

$$U = \sum_{i}^{n} U_i . \qquad [1]$$

O conceito de *utilização* serve de base para testes simples e bastante usados. A idéia central nesses testes é que a soma dos fatores de utilização (U_i) das tarefas do conjunto não ultrapasse o número de processadores disponíveis para suas execuções:

$$U = \sum_{i}^{n} U_i \leq m$$

onde m é o número de processadores. Testes baseados na utilização podem ser exatos, necessários ou suficientes, dependendo da política usada e do modelo de tarefas assumido [Kop92c].

2.4 Escalonamento de Tarefas Periódicas

Nesse item é tratado o problema do escalonamento de tarefas periódicas. Se considerarmos aplicações de tempo real de uma maneira geral, sejam controle de processos ou mesmo aplicações de multimídia, as atividades envolvidas nessas aplicações se caracterizam basicamente pelo comportamento periódico de suas ações. As características de tarefas periódicas que determinam o conhecimento *a priori* dos tempos de chegada e, por conseqüência, da carga computacional do sistema, permitem que se obtenha garantias em tempo de projeto sobre a escalonabilidade de um conjunto de tarefas periódicas.

O escalonamento de tarefas periódicas, nesse texto, é discutido em esquemas

dirigidos a prioridades. Nesses esquemas de escalonamento, as prioridades atribuídas às tarefas do conjunto são derivadas de suas restrições temporais, e não de atributos outros como a importância ou o grau de confiabilidade das tarefas. Escalonamentos baseados em prioridades, além de apresentarem melhor desempenho e flexibilidade, se comparados a abordagens como o executivo cíclico, são objeto de uma literatura relativamente recente e abrangente que estende os trabalhos iniciais introduzidos em [LiL73].

Neste capítulo, é feita uma revisão dos algoritmos de prioridade fixa Taxa Monotônica ("*Rate Monotonic*") [LiL73] e "*Deadline*" Monotônico ("*Deadline Monotonic*") [LeW82] e do algoritmo "*Earliest Deadline First*" [LiL73] que apresenta atribuição dinâmica de prioridades. Esses algoritmos clássicos são tidos como ótimos para suas respectivas classes de problemas, sendo caracterizados por modelos de tarefas simplificados. O conceito de utilização e os testes de escalonabilidade nesses algoritmos são apresentados como forma de análise *a priori* para determinar se existe uma escala realizável que garanta as restrições temporais impostas sobre um conjunto de tarefas periódicas. Na seqüência, após o estudo destes modelos clássicos, são aprofundados os esquemas de prioridade fixa, nas suas extensões aos trabalhos de Liu [LiL73]. Um estudo similar para esquemas de prioridade dinâmica é apresentado no *Anexo A*.

2.4.1 Escalonamento Taxa Monotônica [LiL73]

O escalonamento Taxa Monotônica ("*Rate Monotonic*") produz escalas em tempo de execução através de escalonadores preemptivos, dirigidos a prioridades. É um esquema de prioridade fixa; o que define então, o RM como escalonamento estático e *on-line* segundo os conceitos apresentados no item 2.3.1. O RM é dito ótimo entre os escalonamentos de prioridade fixa na sua classe de problema, ou seja, nenhum outro algoritmo da mesma classe pode escalonar um conjunto de tarefas que não seja escalonável pelo RM.

As premissas do RM que facilitam as análises de escalonabilidade, definem um modelo de tarefas bastante simples :

a. As tarefas são periódicas e independentes.
b. O "*deadline*" de cada tarefa coincide com o seu período (D_i=P_i).
c. O tempo de computação (C_i) de cada tarefa é conhecido e constante ("*Worst Case Computation Time*").
d. O tempo de chaveamento entre tarefas é assumido como nulo.

As premissas *a* e *b* são muito restritivas para o uso desse modelo na prática, contudo essas simplificações são importantes para que se tenha o entendimento sobre o escalonamento de tarefas periódicas.

A política que define a atribuição de prioridades usando o RM, determina uma

ordenação baseada nos valores de períodos das tarefas do conjunto: as prioridades decrescem em função do aumento dos períodos, ou seja, quanto mais freqüente a tarefa maior a sua prioridade no conjunto. Como os períodos das tarefas não mudam, o RM define uma atribuição estática de prioridades (prioridade fixa).

A análise de escalonabilidade no RM, feita em tempo de projeto, é baseada no cálculo da utilização. Para que *n* tarefas tenham o atendimento de suas restrições temporais quando escalonadas pelo RM, deve ser satisfeito o teste abaixo que define uma condição *suficiente*:

$$U = \sum_{i}^{n} C_i / P_i \leq n \left(2^{1/n} - 1\right). \qquad [2]$$

A medida que *n* cresce, nesse teste, a utilização do processador converge para 0,69. Uma utilização de aproximadamente 70% define uma baixa ocupação do processador que, certamente, implica no descarte de muitos conjuntos de tarefas com utilização maior e que, mesmo assim, apresentam escalas realizáveis ("*feasible*").

Essa condição suficiente pode ser relaxada quando as tarefas do conjunto apresentam períodos múltiplos do período da tarefa mais prioritária. Nesse caso a utilização alcançada sob o RM se aproxima do máximo teórico, coincidindo o teste abaixo com uma condição *necessária e suficiente* [Kop92c]:

$$U = \sum_{i}^{n} C_i / P_i \leq 1.$$

Tarefas Periódicas	***Periodo*** (P_i)	***Tempo de Computação*** (C_i)	***Prioridade RM*** (p_i)	***Utilização*** (U_i)
tarefa A	100	20	1	0,2
tarefa B	150	40	2	0,267
tarefa C	350	100	3	0,286

tabela 2.1: Utilização de tarefas periódicas

A tabela 2.1 mostra um exemplo de conjunto de tarefas periódicas onde o objetivo é verificar a escalonabilidade desse conjunto sob a política Taxa Monotônica. A utilização do processador por esse conjunto de tarefas corresponde a 0,753. Aplicando a equação [2] é concluído que esse conjunto é escalonável sob o RM:

$$0{,}753 \leq n \left(2^{1/n} - 1\right) = 0{,}779$$

A figura 2.1 apresenta – na forma de um *diagrama de Gantt* – a escala RM correspondente à tabela 2.1. As tarefas *A*, *B* e *C* chegam em *t=0*. Por ser mais freqüente (maior prioridade segundo o RM), *A* assume o processador. Em *t=20*, a tarefa *A* conclui e *B* toma posse do processador por ser a mais prioritária na fila de Pronto. A tarefa *C*

assume em *t=60* e é interrompida quando da chegada da nova ativação de *A* em *t=100* que, por sua vez, passa a se executar (preempção de *C* por *A*). *C* sofre mais interrupções de novas ativações das tarefas *B* e *A* em *t=150* e *t=200*, respectivamente.

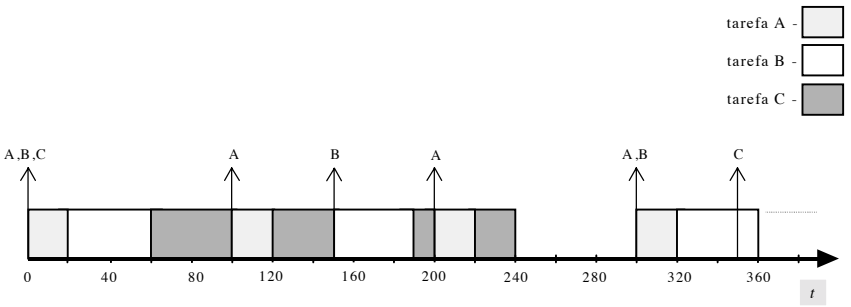


Figura 2.5: Escala RM produzida a partir da Tabela 2.1

2.4.2 Escalonamento "*Earliest Deadline First*" (EDF) [LiL73]

O "*Earliest Deadline First*" (EDF) define um escalonamento baseado em prioridades: a escala é produzida em tempo de execução por um escalonador preemptivo dirigido a prioridades. É um esquema de prioridades dinâmicas com um escalonamento "*on-line*" e dinâmico (item 2.3.1.). O EDF é um algoritmo ótimo na classe dos escalonamentos de prioridade dinâmica. As premissas que determinam o modelo de tarefas no EDF são idênticas às do RM:

a. As tarefas são periódicas e independentes.
b. O "*deadline*" de cada tarefa coincide com o seu período ($D_i=P_i$).
c. O tempo de computação (C_i) de cada tarefa é conhecido e constante ("*Worst Case Computation Time*").
d. O tempo de chaveamento entre tarefas é assumido como nulo.

A política de escalonamento no EDF corresponde a uma atribuição dinâmica de prioridades que define a ordenação das tarefas segundo os seus "*deadlines*" absolutos (d_i). A tarefa mais prioritária é a que tem o "*deadline*" d_i mais próximo do tempo atual. A cada chegada de tarefa a fila de prontos é reordenada, considerando a nova distribuição de prioridades. A cada ativação de uma tarefa T_i, seguindo o modelo de tarefas periódicas introduzido no item 2.2., um novo valor de "*deadline*" absoluto é determinado considerando o número de períodos que antecede a atual ativação (k): $d_{ik}=kP_i$.

No EDF, a escalonabilidade é também verificada em tempo de projeto, tomando como base a utilização do processador. Um conjunto de tarefas periódicas satisfazendo as premissas acima é escalonável com o EDF se e somente se :

$$U = \sum_{i=1}^{n} C_i / P_i \leq 1. \qquad [3]$$

Esse teste é *suficiente* e *necessário* na classe de problema definida para o EDF pelas premissas *a, b, c* e *d*. Se qualquer uma dessas premissas é relaxada (por exemplo, assumindo $D_i \neq P_i$), a condição [3] continua a ser *necessária* porém não é mais *suficiente*.

No exemplo mostrado na figura 2.6, o mesmo conjunto de tarefas é submetido a escalonamentos EDF e RM. A utilização do conjunto de tarefas é de 100%. Com isto pela equação [2] o conjunto não é escalonável pelo RM. Entretanto, a equação [3] garante a produção de uma escala pelo EDF. No caso (b) da figura 2.6, no tempo *t = 50* a tarefa B perde seu "*deadline*". Além da melhor utilização, uma outra diferença, normalmente citada na literatura, é que o EDF produz menos preempções que o RM. A favor do RM está a sua simplicidade que facilita a sua implementação.

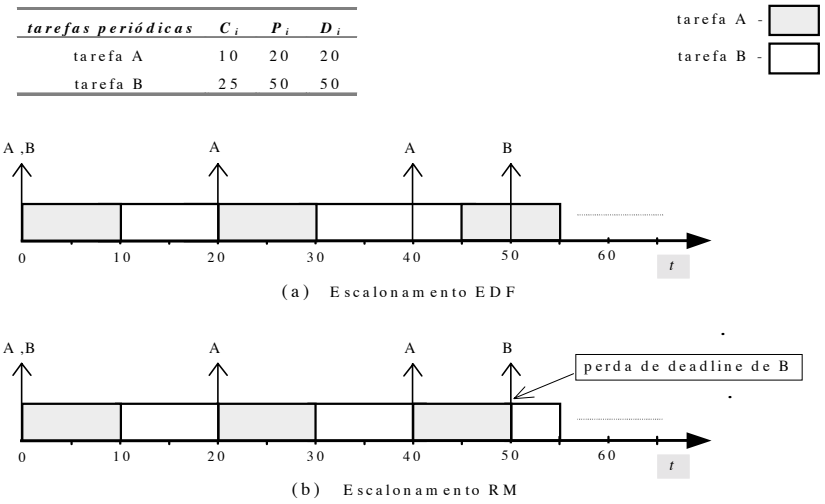


Figura 2.6: Escalas produzidas pelo (a) EDF e (b) RM

2.4.3 Escalonamento "*Deadline*" Monotônico [LeW82]

O "*Deadline Monotonic*" introduzido em [LeW82] estende o modelo de tarefas do Taxa Monotônica. A premissa do RM que limitava os valores de "*deadlines*" relativos aos valores dos períodos das tarefas ($D_i=P_i$), em muitas aplicações, pode ser considerada bastante restritiva. O modelo de tarefas do DM também define tarefas periódicas independentes, o pior caso de tempo de processamento das tarefas (C_i) e o

chaveamento entre tarefas de duração nula. Porém assume "*deadlines*" relativos menores ou iguais aos períodos das tarefas ($D_i \leq P_i$).

A política do DM define uma atribuição estática de prioridades, baseada nos "*deadlines*" relativos das tarefas (D_i). As prioridades são atribuídas na ordem inversa dos valores de seus "*deadlines*" relativos. A produção da escala, portanto, é feita em tempo de execução por escalonador preemptivo dirigido a prioridades. O esquema de prioridades fixas do DM também define um escalonamento estático e "*on-line*". O "*Deadline*" Monotônico é também um algoritmo ótimo na sua classe de problema. A figura 2.7 mostra a escala produzida pelo DM para um conjunto de tarefas periódicas que apresentam "*deadlines*" menores que seus respectivos períodos.

tarefas periódicas	C_i	P_i	D_i	p_i
tarefa A	2	10	6	1
tarefa B	2	10	8	2
tarefa C	8	20	16	3

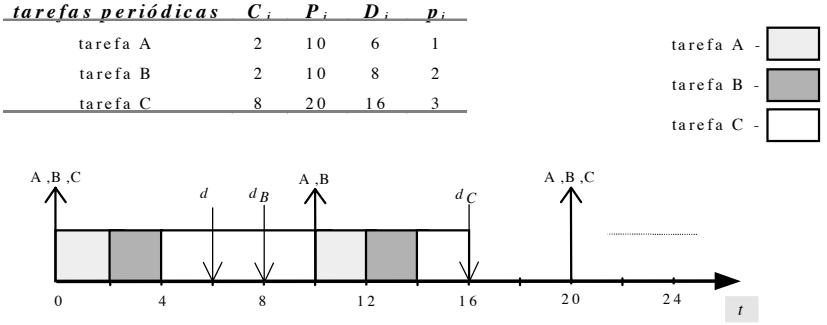


Figura 2.7: Escala produzida pelo DM

Na introdução desse algoritmo os autores não apresentaram em [LeW82] um teste correspondente. Em [ABR91] é definido um teste suficiente e necessário para o DM, baseado no conceito de *tempo de resposta* de uma tarefa. Uma evolução deste teste é descrito na seqüência, no item 2.5.2.

2.5 Testes de Escalonabilidade em Modelos Estendidos

Os testes de escalonabilidade apresentados até aqui são baseados em limites na utilização do processador. Os modelos de tarefas nesses algoritmos preemptivos, baseados em prioridades, são mantidos simples o que facilita as análises nesses testes. Qualquer extensão nos modelos permitindo, por exemplo, que tarefas possam ter relações de precedência entre si ou que os "*deadlines*" assumam valores arbitrários, define modelos mais próximos de aplicações concretas de tempo real. Porém, com essas extensões, os testes fundamentados na noção de utilização, como apresentados anteriormente, passam a ser *condições necessárias* e novos testes mais restritivos são desejáveis.